

Week12_예습과제_권지원

Chapter 08. 텍스트 분석

1. 텍스트 분석 이해

NLP와 텍스트 분석(TA)의 차이

- 머신러닝이 보편화되며 두 영역의 경계가 허물어지고 있으나, 발전의 중점은 다름
- **NLP (Natural Language Processing)**
 - 기계가 인간의 언어를 이해하고 해석하는 데 중점을 둠
 - 예: 기계 번역, 자동으로 질문을 해석하고 답을 해주는 질의응답 시스템
- **텍스트 분석 (Text Analytics / Text Mining)**
 - 비정형 텍스트에서 의미 있는 정보를 추출해 비즈니스 인텔리전스, 예측 분석 등을 수행하는 데 중점을 둠
 - 룰 기반 시스템에서 머신러닝 데이터 기반 모델로 발전하며 기술적 도약이 이루어짐

텍스트 분석의 주요 기술 영역

- **텍스트 분류 (Text Classification / Categorization)**
 - 문서가 특정 분류나 카테고리에 속하는 것을 예측하는 기법 (지도학습 적용)
 - 예: 뉴스 기사의 연예/정치/사회/문화 카테고리 자동 분류, 스팸 메일 검출
- **감성 분석 (Sentiment Analysis)**
 - 텍스트에서 나타나는 주관적인 감정, 판단, 믿음, 기분 등을 분석하는 기법 (지도/비지도학습 모두 적용 가능)
 - 예: 소셜 미디어 감정 분석, 영화/제품 리뷰 긍정 및 부정 분석, 여론조사 의견 분석
- **텍스트 요약 (Summarization)**
 - 텍스트 내에서 중요한 주제나 중심 사상을 추출하는 기법
 - 예: 토픽 모델링 (Topic Modeling)
- **텍스트 군집화 (Clustering) 및 유사도 측정**
 - 비슷한 유형의 문서들을 비지도학습 기반으로 군집화하거나 문서 간의 유사도를 측정해 모으는 기법

텍스트 분석 수행 프로세스

1. **텍스트 사전 준비작업 (텍스트 전처리):** 클렌징, 대/소문자 변경, 특수문자 삭제, 토큰화, 스톱 워드 제거, 어근 추출(Stemming/Lemmatization) 등 텍스트 정규화 수행
2. **피처 벡터화/추출:** 사전 준비된 텍스트에서 피처를 추출하고 숫자형 벡터 값을 할당 (BOW의 Count 및 TF-IDF, Word2Vec 등)
3. **ML 모델 수립 및 학습/예측/평가:** 피처 벡터화된 데이터 세트에 머신러닝 알고리즘 적용

대표적인 파이썬 NLP 패키지

패키지	특징
NLTK (Natural Language Toolkit)	방대한 데이터 세트와 서브 모듈을 갖춘 파이썬의 가장 대표적인 NLP 패키지. 수행 속도 측면에서 아쉬움이 있어 대량의 데이터 처리에는 한계가 있음
Gensim	토픽 모델링 분야에서 가장 두각을 나타내는 패키지. Word2Vec 구현 등 다양한 신기능을 제공하며 실무에서 많이 사용됨
SpaCy	뛰어난 수행 성능으로 최근 가장 주목받는 패키지. 많은 NLP 애플리케이션에서 도입이 늘고 있음

2. 텍스트 사전 준비 작업 (텍스트 정규화)

클렌징 (Cleansing)

- 분석에 방해가 되는 불필요한 문자, 기호 등을 사전에 제거 (예: HTML, XML 태그 등)

텍스트 토큰화 (Tokenization)

- 문장 토큰화:** 마침표(.), 개행문자(\n) 등 문장의 마지막을 뜻하는 기호나 정규 표현식에 따라 문장을 분리 (NLTK의 `sent_tokenize` 활용)
 - 각 문장이 가지는 시맨틱적인(문맥적) 의미가 중요할 때 주로 사용함
- 단어 토큰화:** 공백, 콤마, 마침표, 개행문자 등으로 문장을 단어로 분리 (NLTK의 `word_tokenize` 활용)
- n-gram:** 문장을 단어별로 하나씩 토큰화할 때 발생하는 문맥적 의미 무시 문제를 해결하기 위해 도입
 - 연속된 n개의 단어를 하나의 토큰화 단위로 묶어 순차적으로 이동하며 토큰화 수행 (예: bigram)

스톱 워드 (Stop word) 제거

- 영어의 is, the, a처럼 문법적 필수 요소지만 분석에는 의미가 없는 단어를 제거하는 작업
- 빈번하게 나타나기 때문에 제거하지 않으면 중요한 단어로 잘못 인지될 수 있음
- 언어별로 목록화되어 있으며 NLTK에서 다양한 언어의 스톱 워드 목록을 제공함

어근 추출 (Stemming vs Lemmatization)

- 문법적 또는 의미적으로 변화하는 단어(예: work, working, worked)의 원형을 찾는 작업

구분	특징	NLTK 지원 클래스
Stemming	일반적이거나 단순화된 룰을 적용하여 철자가 일부 훼손된 어근 단어를 추출함 (변환 속도가 빠름)	<code>LancasterStemmer</code> , <code>Porter</code> , <code>Snowball</code>
Lemmatization	품사와 같은 문법적 요소와 의미론적 기반을 모두 감안하여 정확한 철자의 어근 단어를 찾음 (시간이 더 소요되며, 변환 시 품사 정보를 입력해 주어야 정확도가 높음)	<code>WordNetLemmatizer</code>

3. Bag of Words (BOW)와 피처 벡터화

BOW (Bag of Words) 개념

- 문서가 가지는 모든 단어를 문맥이나 순서를 무시하고 일괄적으로 봉투(Bag)에 넣어 단어 빈도 값을 부여해 피처를 추출하는 모델

- **장점:** 구축이 쉽고 빠르며, 문서의 특징을 잘 나타내어 전통적으로 활용도가 높음
- **단점:**
 - **문맥 의미 반영 부족:** 단어의 순서를 고려하지 않아 문맥적인 해석이 어려움 (n-gram으로 일부 보완 가능하나 한계 존재)
 - **희소 행렬(Sparse Matrix) 문제:** 수만~수십만 개의 단어 칼럼 중 개별 문서에 포함된 단어는 극히 일부이므로, 대부분의 값이 0으로 채워짐. 이는 학습 시간 증가와 예측 성능 저하를 유발함

BOW 피쳐 벡터화 방식

벡터화 방식	설명	사이킷런 클래스
Count 벡터화	단어가 문서에 나타나는 횟수(Count)를 값으로 부여. 범용적으로 자주 쓰이는 단어까지 중요하게 인식되는 문제 발생	<code>CountVectorizer</code>
TF-IDF 벡터화	자주 나타나는 단어에 높은 가중치를 주되, 모든 문서에서 전반적으로 자주 나타나는 범용적 단어 (예: '많은', '당연히')에는 페널티를 주는 방식	<code>TfidfVectorizer</code>

CountVectorizer / TfidfVectorizer 주요 파라미터

파라미터	설명
max_df	빈도수가 너무 높은 단어를 제외 (예: 100 지정 시 100개 이하 출현 단어만 추출, 0.95 지정 시 상위 5% 제외)
min_df	빈도수가 너무 낮은 가비지성 단어를 제외 (예: 2 지정 시 2번 이하 출현 단어 제외, 0.02 지정 시 하위 2% 제외)
max_features	추출하는 피쳐의 최대 개수를 제한 (가장 높은 빈도수를 가지는 단어 순으로 추출)
stop_words	'english' 지정 시 내장된 영어 스톱 워드를 추출에서 제외
n_gram_range	단어 순서를 보장하기 위한 튜플 (최솟값, 최댓값). 예: (1, 2) 는 단어 1개씩(unigram) 및 2개씩 묶어서(bigram) 추출
tokenizer	토큰화를 수행하는 별도의 커스텀 함수(예: 어근 추출 결합)를 적용할 때 사용

희소 행렬 (Sparse Matrix) 최적화 형식

- 많은 메모리 공간을 차지하고 연산 시간을 낭비하는 불필요한 0 값을 효율적으로 저장하는 방법

형식	설명
COO 형식 (Coordinate)	0이 아닌 데이터만 별도의 배열에 저장하고, 그 데이터가 가리키는 행(Row)과 열(Column)의 위치를 각각 별도의 배열로 저장하는 방식 (Scipy의 <code>coo_matrix</code> 활용)
CSR 형식 (Compressed Sparse Row)	COO 형식에서 '행 위치 배열'의 값이 반복적으로 나타나는 문제를 해결. 고유한 값의 '시작 인덱스'만을 새로운 배열로 저장하여 메모리를 더 줄이고 연산 속도를 높인 방식 (Scipy의 <code>csr_matrix</code> 활용, 사이킷런 기본 변환 형태)

4. 텍스트 분류 (Text Classification) 실습 - 20 뉴스그룹

분류 수행 핵심 프로세스 및 주의점

- **데이터 로딩:** `fetch_20newsgroups()` API를 이용해 데이터 다운로드 및 로딩 수행

- 데이터 정제 (헤더/푸터 제거):

- 뉴스그룹 데이터의 헤더, 푸터, 작성자 정보 등은 타깃 클래스와 직접적인 연관성이 너무 높아 모델 성능을 비정상적으로 높일 수 있음
- 순수한 텍스트 분석을 위해 `remove=('headers', 'footers', 'quotes')` 파라미터를 사용해 기사 내용만 추출해야 함

- 학습/테스트 데이터 변환 시 주의점 (매우 중요):

- 테스트 데이터에 피쳐 벡터화를 적용할 때는 반드시 학습 데이터로 `fit()` 이 수행된 변환기 (Vectorizer) 객체를 이용해 `transform()` 만 수행해야 함
- 테스트 데이터에 다시 `fit_transform()` 을 적용하면 피쳐 공간(칼럼 수)이 학습 데이터와 달라져 모델 예측이 불가능해짐

파이프라인(Pipeline)과 GridSearchCV 결합

- 데이터 전처리(피쳐 벡터화)와 ML 모델(로지스틱 회귀 등) 학습/예측 과정을 '수도관에서 물이 흐르듯' 한 번에 스트림 기반으로 처리하는 객체
- 코드가 직관적이게 되며 수행 시간을 절약할 수 있음
- `GridSearchCV` 에 파이프라인을 적용해 하이퍼 파라미터 튜닝을 동시에 수행할 때의 명명 규칙:
 - 개별 객체 변수명 뒤에 언더바 2개(`__`)를 연달아 붙이고 파라미터명을 결합해 Key 값으로 할당
 - 예: `tfidf_vect_ngram_range` , `lr_clf_C`
 - 단점: 튜닝 경우의 수가 급격히 많아져 학습 및 최적화 시간이 매우 오래 걸릴 수 있음

5. 감성 분석 (Sentiment Analysis)

지도학습 기반 감성 분석

- 학습 데이터와 타깃 레이블(긍정 1, 부정 0) 값을 기반으로 모델을 학습한 뒤 새로운 텍스트의 감성을 예측하는 텍스트 이진 분류 방식
- IMDB 영화 리뷰 실습: 정규 표현식(`re.sub`)을 이용해 특수문자나 `<br />` 태그 등을 공백으로 치환하는 클렌징 작업 수행 후 파이프라인(TF-IDF + 로지스틱 회귀)을 적용해 분류 예측

비지도학습 기반 감성 분석

- 데이터에 레이블이 없는 경우, 감성 어휘 사전(Lexicon)을 활용하여 긍정적/부정적 감성 여부를 판단

주요 감성 사전 (Lexicon)

사전 이름	특징 및 작동 원리
SentiWordNet	NLTK 패키지의 WordNet 기반. 방대한 시맨틱(문맥상 의미) 정보를 담은 <code>Synset</code> 개념을 활용함. 단어별로 긍정 감성 지수, 부정 감성 지수, 객관성 지수를 할당하며 예측 성능 자체는 실무 적용에 다소 무리가 있음
VADER	소셜 미디어 텍스트에 특화된 룰 기반 사전. 대용량 텍스트에 대해 빠른 수행 시간과 훌륭한 예측 성능을 제공함. <code>SentimentIntensityAnalyzer</code> 클래스의 <code>polarity_scores()</code> 를 호출하여 도출된 <code>compound</code> 점수가 특정 임계값(통상 0.1) 이상이면 긍정으로 판단

6. 토픽 모델링 (Topic Modeling)

토픽 모델링의 개념

- 방대한 문서 집합에 숨어 있는 핵심 주제(Topic)를 찾아내어 함축적인 중심 단어들로 표현하는 기법
- 대표 알고리즘: **LDA (Latent Dirichlet Allocation)**
 - (주의) 차원 축소 기법인 LDA(Linear Discriminant Analysis)와 약어만 동일하며 완전히 다른 알고리즘임

구현 특징 및 실습 (20 뉴스그룹)

- 사이킷런의 `LatentDirichletAllocation` 클래스를 활용
- **피처 벡터화 제약:** LDA 알고리즘은 단어의 빈도수에 기반하므로 반드시 `CountVectorizer`(카운트 기반)만 적용해야 함
- **결과 해석:** 학습 완료 후 `components_` 속성을 통해 각 토픽(행)별로 모든 단어 피처(열)가 해당 토픽에 할당된 연관도 수치 확인 가능
- `argsort()[::-1]` 을 활용해 값이 큰 순서(연관도가 높은 단어)대로 인덱스를 추출하여 토픽별 핵심 단어를 시각화함

7. 문서 군집화 (Document Clustering)

문서 군집화 개념

- 비슷한 텍스트 구성을 가진 문서들을 사전에 결정된 카테고리 값(레이블) 없이 비지도학습 기반으로 군집화(Clustering)하는 기법
- 텍스트 분류와 목적은 유사하나 학습 데이터 세트가 필요 없는 점이 차이임

실습 및 핵심 단어 추출 (Opinion Review)

- 여러 파일로 분산된 리뷰 텍스트를 디렉터리 순회를 통해 하나의 `DataFrame`으로 로드
- `TfidfVectorizer` 를 이용해 문서를 피처 벡터화한 후 `KMeans` 군집화 알고리즘 적용
- **군집별 핵심 단어 파악:**
 - `KMeans` 객체의 `cluster_centers_` 속성은 (군집 개수, 전체 피처 개수) 형태의 배열로, 각 피처가 군집 중심점(Centroid)을 기준으로 얼마나 가깝게 위치하는지 0~1 사이의 상대 좌표 값으로 제공
 - 값이 1에 가까울수록 해당 군집을 대표하는 핵심 단어임
 - 배열 정렬 기법(`argsort()[::-1]`)을 통해 값이 가장 큰 피처 인덱스를 추출하여 군집별 중심 단어 목록을 식별

8. 문서 유사도

코사인 유사도 (Cosine Similarity)

- 벡터의 크기보다는 상호 방향성이 얼마나 유사한지에 기반하여 두 벡터 사이의 사잇각을 수치로 환산한 지표
- 공식 (두 벡터 A와 B의 내적을 총 벡터 크기의 합으로 정규화):

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{|A| |B|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- 코사인 유사도가 텍스트 분석에 적합한 이유:

- 문서를 피쳐 벡터화하면 차원이 매우 많은 희소 행렬이 됨. 희소 행렬 환경에서는 유클리드 거리 기반 지표의 정확도가 떨어짐
- 특정 문서의 길이가 비정상적으로 길어 단어 빈도수가 압도적으로 많아지는 왜곡(크기 차이) 현상을 상쇄하고 공정한 비교를 가능하게 함

- 사이킷런의 `sklearn.metrics.pairwise.cosine_similarity` API를 통해 희소 행렬 형태를 굳이 밀집 행렬로 변환하지 않고도 직접 문서 간 유사도 쌍(pair)을 연산할 수 있음

9. 한글 텍스트 처리 (KoNLPy)

한글 NLP 처리의 어려움

- **띄어쓰기:** 띄어쓰기를 잘못하면 없는 단어로 인식되는 영어나 라틴어와 달리, 한글은 의미 전체가 완전히 왜곡되어 전달될 위험이 큼
- **다양한 조사:** 주어, 목적어 등을 나타내는 조사의 경우의 수가 워낙 많아 어근 추출 등 전처리 단계에서 분리하고 제거하기가 매우 까다로움

KoNLPy (코엔엘파이) 소개

- 파이썬의 대표적인 한글 형태소 분석(Morphological analysis) 패키지
- 꼬꼬마(Kkma), 한나눔(Hannanum), Komoran, 은전한닢(Mecab), Twitter(Okt) 등 기존 C/C++/Java로 작성된 뛰어난 형태소 엔진을 파이썬 래퍼(Wrapper) 형태로 제공
- **설치 주의사항:**
 - Java 기반 엔진을 구동하기 때문에 OS 비트 수에 맞는 Java JDK(1.7 이상) 설치가 선행되어야 함
 - 파이썬에서 Java 클래스를 호출하기 위한 `JPyype1` 패키지 설치 필요
 - 윈도우 환경에서는 일반적인 JDK 폴더가 아닌 `jvm.dll` 이 위치한 경로(예: `\bin\server`)로 `JAVA_HOME` 환경 변수를 설정해야 정상 구동됨

10. 텍스트 분석 실습 — 캐글 Mercari Price Suggestion

비정형 텍스트와 정형 속성이 혼합된 데이터의 가격 예측(회귀) 과제

데이터 전처리 및 Target 값 변환

- **결측치(Null) 처리:** 브랜드명 등 텍스트 칼럼의 Null 값은 별도 처리가 어려우므로 문자열 `'Other_Null'` 로 일괄 치환
- **타겟 변수(Price) 정규화:**
 - 가격 데이터 분포가 한쪽으로 극심하게 왜곡(Skew)되어 있으므로 `np.log1p()` 를 적용해 로그 값으로 변환하여 정규 분포 형태로 유도
 - 예측 평가(RMSLE) 시에는 예측된 로그 값을 다시 `np.expml()` 을 통해 역변환(지수 변환)하여 원래 가격 수준으로 원복 후 측정해야 함

피처 인코딩과 피처 벡터화의 분리 및 결합

- 텍스트 길이와 특징에 따른 피처 벡터화:

- 상품명(Name): 적은 단어 위주의 짧은 텍스트이므로 `CountVectorizer` 적용
- 상품 설명(Item_description): 평균 문장 길이가 긴 텍스트이므로 `TfidfVectorizer` 적용 (파라미터 `max_features` 로 피처 수 제한)

- 정형 데이터(문자열 범주형)의 인코딩:

- 브랜드명, 카테고리 등 범주형 데이터는 원-핫 인코딩 적용
- 벡터화된 텍스트(희소 행렬)와 형태를 맞추기 위해, `LabelBinarizer(sparse_output=True)` 를 적용하여 처음부터 인코딩 결과를 **희소 행렬 형태**로 생성

- 행렬 결합:

- `scipy.sparse.hstack` 모듈을 이용해 피처 벡터화된 행렬들과 원-핫 인코딩된 희소 행렬들을 칼럼(가로) 방향으로 모두 결합하여 거대한 하나의 학습 데이터 세트로 구성

모델 구축 및 평가

- 메모리를 매우 많이 차지하므로 결합이 끝난 중간 객체는 `gc.collect()` 와 `del` 명령어로 바로바로 메모리에서 해제
- 원-핫 인코딩과 텍스트 벡터 행렬에 성능이 좋은 선형 회귀 모델(Ridge)과 트리 기반 모델(LightGBM)을 각각 학습시키고, 두 예측값을 혼합하는 간단한 **앙상블(Ensemble)** 방식을 통해 단일 모델보다 최종 예측 성능(RMSLE)을 향상시킴